AuraMart — System Design Document

Target scale: 10,000 requests/second at peak · 99.9% availability · p99 < 300 ms for reads **Version:** 1.0 · **Author:** Chinmaya Sutar · **Status:** Design

1. Overview

AuraMart is an event-driven e-commerce platform with an AI shopping assistant. Shoppers browse and search a catalogue, manage a cart, check out through an asynchronous saga, and chat with a grounded AI assistant. Admins manage products and stock.

This document describes how the system is built to handle **10,000 RPS at peak** with high availability, data consistency where it matters, and graceful degradation everywhere else.

2. Requirements

2.1 Functional

#	Requirement
F1	Browse products by category with paging and sorting
F2	Keyword search (typo-tolerant) and semantic search
F3	Cart: add, update, remove, clear
F4	Checkout: reserve stock → payment → confirm or cancel (with compensation)
F5	Order history and live status
F6	AI assistant: grounded product suggestions, order lookup, streaming replies
F7	Email notifications on order confirmation or cancellation
F8	Admin: product CRUD, stock management
F9	Register, login, roles (shopper, admin)

2.2 Non-functional

Attribute	Target
Peak throughput	**10,000 RPS** (sustained 15 min, e.g. sale events)
Average throughput	2,500 RPS
Latency	p50 < 50 ms, p99 < 300 ms (reads); p99 < 800 ms (writes)
AI first token	< 1.5 s
Availability	99.9% (≈ 43 min downtime/month)
Consistency	**Strong** for stock and payment; **eventual** for catalogue, search index, notifications
Durability	Zero order loss (RPO = 0 for orders), RTO < 15 min
Security	JWT auth, TLS everywhere, OWASP Top 10, no secrets in code

2.3 Out of scope

Real payment gateway (simulated), multi-currency, multi-region active-active, seller marketplace.

3. Capacity Estimation

3.1 Traffic mix at 10,000 RPS peak

Request type	Share	RPS	Served by
Product list / detail	55%	5,500	catalog (≈95% from cache)
Keyword search	15%	1,500	catalog / search index
Semantic search	3%	300	ai-assistant (pgvector)
Cart read / write	17%	1,700	order-service
Order status poll	6%	600	order-service (replica)
Place order	1.5%	150	order-service → saga
Auth (login / refresh)	2%	200	auth-service
AI chat	0.5%	50	ai-assistant → LLM

Read : write ≈ 90 : 10. The design optimises for reads.

3.2 Volume

Item	Estimate
Daily active users	2 M
Orders / day	avg 30/s × 86,400 ≈ **2.6 M**
Products	1 M SKUs
Order row + items	~2 KB → **≈ 5 GB/day**, ~1.8 TB/year
Product row	~2 KB → 2 GB total
Product embeddings	1 M × 768 dims × 4 B ≈ **3 GB** (+ HNSW index ≈ 2×)
Kafka events	~6 per order → 900 msg/s peak, ~1 KB each ≈ **1 MB/s**
Logs	10k RPS × 8 hops × 500 B ≈ 40 MB/s peak → sample + 7-day retention

3.3 Cache sizing

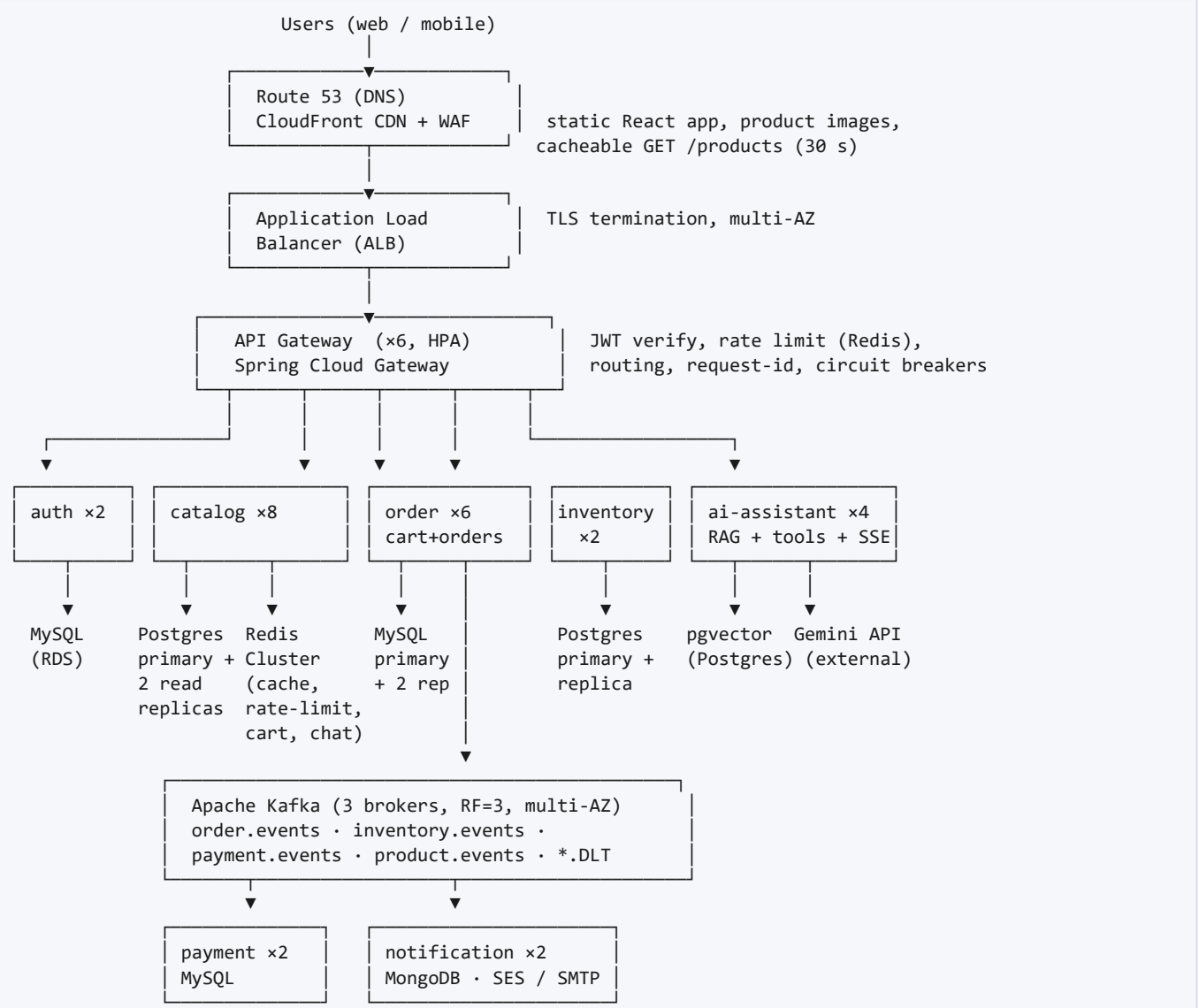
- Hot set ≈ 20% of products → 200k × 2 KB ≈ **400 MB**. Add search results and list pages: **~2 GB of Redis** is enough.
- With a 95% hit rate, DB reads drop from 7,000 to **~350 RPS**.

3.4 Compute sizing (Java 21 + virtual threads)

Component	Throughput / pod	Peak need	Pods (with 50% headroom)
API Gateway	~3,000 RPS	10,000	**6**
catalog-service	~1,500 RPS (cached)	7,000	**8**
order-service	~600 RPS	2,450	**6**
auth-service	~800 RPS (BCrypt-bound)	200	2
inventory-service	~500 events/s	300	2
payment-service	~500 events/s	150	2
notification-service	~300 events/s	300	2
ai-assistant-service	~25 concurrent streams	50 chats + 300 sem.	**4**

Each pod: 1 vCPU / 2 GB RAM. Minimum 2 pods per service in different availability zones.

4. High-Level Architecture



5. Service Design

Service	Responsibility	Store	Sync API	Events
api-gateway	Edge: auth, routing, rate limit	Redis	—	—
auth-service	Users, roles, JWT issue/refresh	MySQL	`/auth/*`	`UserRegistered`
catalog-service	Products, categories, keyword search	PostgreSQL + Redis + S3	`/products/*`	`ProductCreated/Updated/Deleted`
inventory-service	Stock, reservations	PostgreSQL	`/inventory/*` (admin), internal `GET /stock/{id}`	consumes `OrderPlaced`, `OrderCancelled`; emits `StockReserved/Rejected/Released`
order-service	Cart, orders, saga state	MySQL + Redis (cart)	`/carts/*`, `/orders/*`	emits `OrderPlaced/Confirmed/Cancelled` (via outbox)
payment-service	Simulated payment	MySQL	—	consumes `StockReserved`; emits `PaymentSucceeded/Failed`

Service	Responsibility	Store	Sync API	Events
notification-service	Emails, notification log	MongoDB	—	consumes order outcome events
ai-assistant-service	RAG, tools, memory, streaming	pgvector + Redis	`/assistant/*`, `/products/semantic-search`	consumes `product.events`

Rule: each service owns its own database. No service reads another's tables, and there are no cross-service joins.

---

6. Key Flows

6.1 Browse product (hot path — 55% of traffic)

```
Client → CDN (hit? return, ~40% of list pages)
       → ALB → Gateway (JWT optional, rate limit)
       → catalog: Redis GET product:{id}
            hit  (95%) → return               ~5 ms
            miss       → Postgres READ REPLICA → SET with TTL 10 min + jitter → return
```

Writes (admin) go to the primary, publish `ProductUpdated`, and **evict** the cache key and CDN path.

6.2 Add to cart

```
Gateway → order-service
        → Feign: catalog GET /products/{id}   (cached; circuit breaker, 300 ms timeout)
        → Feign: inventory GET /stock/{id}    (soft check only, not a reservation)
        → Redis HSET cart:{customerId}  (TTL 30 days) + async write-behind to MySQL
```

The cart is kept in **Redis** for speed. The real stock decision happens only at checkout.

6.3 Checkout saga (choreography, strongly consistent stock)

```
1. POST /orders  (Idempotency-Key header)
   order-service, ONE DB transaction:
       INSERT orders(status=PENDING) + INSERT outbox(OrderPlaced)
   → 201 PENDING  (response in < 100 ms; client polls)

2. Outbox relay (or Debezium CDC) → Kafka order.events  key=orderId

3. inventory-service consumes OrderPlaced:
       UPDATE stock SET reserved = reserved + :q, version = version + 1
       WHERE product_id = :p AND available - reserved >= :q AND version = :v
       → StockReserved | StockRejected        (idempotent by eventId)

4. payment-service consumes StockReserved → PaymentSucceeded | PaymentFailed

5. order-service:
       PaymentSucceeded → CONFIRMED
       PaymentFailed    → CANCELLED(PAYMENT_FAILED) → OrderCancelled
       StockRejected    → CANCELLED(OUT_OF_STOCK)

6. inventory consumes OrderCancelled → release reservation   (compensation)
7. notification consumes Confirmed/Cancelled → email (retry ×3 → DLT)
```

Why a saga: no distributed 2-phase commit across 3 databases. Each step is local and atomic, and failures are undone by compensating events.

Hot-product contention (flash sale): a single SKU can receive thousands of reservations per second. Mitigations: - The Kafka key is `orderId`, but inventory **re-keys by `productId`** onto an internal topic, so one partition processes one SKU serially, which removes lock contention. - Optional: keep a Redis `DECRBY stock:{sku}` gate in front of the DB for very hot SKUs (Lua script, atomic), and reconcile with the DB.

6.4 AI chat (streaming)

```
Gateway (separate rate limit: 10 msgs/min/user) → ai-assistant
  → load memory from Redis (conversationId)
  → embed query → pgvector top-k (HNSW, ef_search=40) → products as context
  → Gemini streamCompletion with tools [searchProducts, getOrderStatus]
       tool call → Feign → catalog / order
  → SSE: meta → tool → token… → done
  → save memory (TTL 24 h)
```

LLM calls are the most expensive and slowest dependency. They get a **bulkhead** (max concurrent calls), a timeout and a fallback ("Assistant is busy — try search").

---

7. Data Design

7.1 Database choices

Data	Store	Why
Products, categories	PostgreSQL	Relational, `pg_trgm` fuzzy search, JSONB attributes
Stock	PostgreSQL	Row-level locking, strong consistency
Orders, payments, users	MySQL	Transactional, mature replication (also a resume skill)
Cart, sessions, rate limits, chat memory	Redis	Sub-ms, TTL-based, high write rate
Notification log	MongoDB	Schemaless, append-heavy, flexible payloads
Embeddings	PostgreSQL + pgvector	Vector search next to relational filters (price, category)
Images	S3 + CloudFront	Cheap, durable, edge-cached

7.2 Core schemas (simplified)

```
 -- catalog
products(id UUID PK, sku VARCHAR UNIQUE, name, description, category, price NUMERIC(12,2),
         currency CHAR(3), attributes JSONB, image_key, active BOOL,
         version INT, created_at, updated_at)
INDEX (category, active, price);  GIN (name gin_trgm_ops);

-- inventory
stock(product_id UUID PK, available INT CHECK (available >= 0), reserved INT, version INT)
reservations(order_id, product_id, qty, status, PRIMARY KEY(order_id, product_id))
processed_events(event_id UUID PK, processed_at)

-- order
orders(id UUID PK, customer_id UUID, status, total, currency, shipping_address JSON,
       idempotency_key UNIQUE, created_at, updated_at)   INDEX (customer_id, created_at DESC)
order_items(order_id, product_id, name, unit_price, qty)
order_timeline(order_id, event, at)
outbox(id UUID PK, aggregate_id, type, payload JSON, created_at, published_at NULL)

-- ai
product_embeddings(product_id UUID PK, embedding VECTOR(768), content TEXT, updated_at)
INDEX USING hnsw (embedding vector_cosine_ops);
```

7.3 Scaling the data tier

Technique	Applied to
Read replicas (2 per primary)	catalog, order — all list/detail/poll reads
Connection pooling	HikariCP, max 20 per pod; **RDS Proxy / PgBouncer** in front
Partitioning	`orders` partitioned by month (range on `created_at`)
Sharding (future)	`orders` by `hash(customer_id)` once > 5 TB or > 5k writes/s

Technique	Applied to
Archiving	Orders > 1 year → S3 (Parquet)
Indexes	Every query path has an index; checked with `EXPLAIN ANALYZE`

8. Caching Strategy

Layer	What	TTL	Invalidation
Browser	Static assets (hashed filenames)	1 year	New filename on deploy
CDN (CloudFront)	React app, images, `GET /products?…` anonymous	30 s	Short TTL + invalidation on admin write
Redis	`product:{id}`, `products:list:{hash(params)}`, `search:{q}`	10 min ± jitter	`@CacheEvict` on `ProductUpdated` event
Redis	`cart:{customerId}`	30 days	Source of truth for cart
Local (Caffeine)	Categories, config	60 s	Time-based

Cache problems and their fixes: - **Stampede** (a hot key expires and 1,000 requests hit the DB): single-flight lock (`SET NX`) plus early refresh. - **Penetration** (requests for IDs that don't exist): cache the negative result for 60 s; validate UUIDs at the edge. - **Avalanche** (many keys expire together): add a random jitter to the TTL.

9. Messaging (Kafka)

Topic	Partitions	Key	Producers → Consumers	Retention
order.events	12	orderId	order → inventory, payment, notification, ai	7 d
inventory.events	12	orderId	inventory → order	7 d
inventory.reservations	24	productId	inventory internal (serialises hot SKUs)	1 d
payment.events	12	orderId	payment → order	7 d
product.events	6	productId	catalog → ai, catalog cache evict	3 d (compacted)
*.DLT	3	—	failed messages	30 d

Settings: 3 brokers across AZs, `replication.factor=3`, `min.insync.replicas=2`, producer `acks=all`, `enable.idempotence=true`. **Delivery guarantee:** at-least-once, made safe by **idempotent consumers** (a `processed_events` table, or a unique key per event). **Ordering:** guaranteed per key (per order, or per product).

10. Scalability

Concern	Solution
Stateless services	All state in DB, Redis or Kafka, so any pod can serve any request
Horizontal scaling	Kubernetes **HPA** on CPU 60% and RPS; Kafka consumers scale up to the partition count
Edge offload	CDN serves static content plus ~40% of anonymous catalogue reads
Thread model	Java 21 **virtual threads** (`spring.threads.virtual.enabled=true`) for blocking I/O
Async writes	Checkout returns 201 immediately; the heavy work happens in the saga
Database	Replicas for reads, pooling, partitioning, sharding path documented
Hot keys	Re-key by productId, Redis atomic counters
Expensive AI	Separate pool, bulkhead, per-user quota, cached embeddings for frequent queries

11. Reliability & Fault Tolerance

Failure	Behaviour
catalog DB primary down	RDS Multi-AZ failover (< 60 s); reads keep working from replicas and cache
Redis down	Cache falls back to DB (circuit breaker); rate limiter fails **open**
Kafka broker down	RF=3 + `min.insync=2`, so no data loss; the outbox keeps events until Kafka is back
inventory slow	Saga waits; order stays PENDING; consumer lag alert fires
payment never responds	**Saga timeout** (60 s scheduler) → cancel + release stock
Notification fails	Retry 1 s → 10 s → 60 s → DLT, with an alert and a replay tool
Gemini down / slow	Timeout 10 s → fallback message; shopping is unaffected
Downstream Feign call fails	Resilience4j: timeout 300 ms, 2 retries with jitter, circuit breaker (50% / 20 calls), fallback
Duplicate request	`Idempotency-Key` on `POST /orders`, unique constraint
Poison message	Fails deserialization → straight to DLT
AZ outage	Every tier spread over 3 AZs; ALB routes around

Graceful degradation order (shed first → protect last): AI assistant → semantic search → recommendations → keyword search → browse → **checkout (protected)**.

12. Security

Area	Control
Edge	AWS WAF (SQLi/XSS rules, bot control), CloudFront, TLS 1.2+
Authentication	JWT (RS256) access 15 min + rotating refresh 7 d; or Keycloak OIDC + PKCE
Authorization	Gateway verifies the token; services enforce roles with `@PreAuthorize`
Service-to-service	Private subnets, security groups, mTLS (service mesh, optional)
Rate limiting	Redis token bucket: 100 req/s per IP anonymous, 50 req/s per user, 10 AI msgs/min
Secrets	AWS Secrets Manager / env vars; IAM roles for S3 (no static keys)
Data	Encryption at rest (RDS, S3, EBS), BCrypt passwords, no PII in logs
Input	Bean Validation, size limits, parameterised queries
AI	Prompt-injection guard, tools run as the **calling user** only, output rendered as safe Markdown
Supply chain	Dependabot, OWASP dependency-check, Trivy image scan in CI

13. Observability

Pillar	Tooling	Key signals
Logs	Logback JSON → Fluent Bit → OpenSearch / **Kibana**	`traceId`, `requestId`, `customerId`, `orderId`
Metrics	Micrometer → Prometheus → Grafana	RED per endpoint, JVM, Hikari pool, cache hit %, Kafka lag
Traces	OpenTelemetry → Tempo / Jaeger	Full path Gateway → order → Kafka → inventory → payment
Alerts	Alertmanager → Slack / email	see SLOs

SLOs and alerts

SLO	Target	Alert when
Read availability	99.9%	5xx > 0.5% for 5 min
Read latency	p99 < 300 ms	p99 > 500 ms for 10 min
Order completion	99% reach a final state within 10 s	PENDING > 60 s count > 50

SLO	Target	Alert when
Kafka consumer lag	< 1,000 messages	lag > 5,000 for 5 min
Cache hit rate	> 90%	< 80% for 10 min
DLT	0 messages	any message

14. Deployment Architecture

```
AWS Region (ap-south-1, Mumbai)
├── VPC 10.0.0.0/16, 3 AZs
│   ├── Public subnets:  ALB, NAT gateways
│   ├── Private app:     EKS node groups (m6i.large, autoscaling 6–30 nodes)
│   └── Private data:    RDS MySQL & Postgres (Multi-AZ + replicas),
│                        ElastiCache Redis (cluster, 3 shards × 1 replica),
│                        MSK Kafka (3 brokers), DocumentDB/MongoDB Atlas
├── S3: product images, order archive, backups
├── CloudFront + WAF + Route 53
└── Secrets Manager, ECR, CloudWatch
```

CI/CD (GitHub Actions): PR → build, unit + Testcontainers tests, coverage gate, SAST, image scan → merge → push to ECR → deploy to **staging** → smoke + k6 load test → manual approve → **blue/green or canary (10% → 100%)** in production, with auto-rollback on SLO breach.

Learning version: the same containers on one EC2 instance with Docker Compose. The architecture stays the same; only the infrastructure is smaller.

15. Load-Testing Plan (proving 10k RPS)

Test	Tool	Shape	Pass criteria
Baseline	k6	1k RPS, 10 min	p99 < 200 ms, 0 errors
Peak	k6 (distributed)	ramp to 10k RPS in 5 min, hold 15 min	p99 < 300 ms, errors < 0.1%
Spike	k6	2k → 15k RPS in 30 s	HPA reacts < 2 min, no 5xx storm, rate limiter returns 429
Soak	k6	3k RPS, 8 h	no memory leak, stable latency
Flash sale	k6	500 orders/s on 1 SKU with 100 units	**exactly 100 confirmed**, no overselling
Chaos	Chaos Mesh / manual	kill a pod, a broker, the Redis primary	SLOs hold, no lost orders

16. Trade-offs & Decisions

Decision	Chosen	Alternative	Why
Inter-service style	Async events for checkout, sync Feign for reads	All sync	Decoupling and resilience where it matters
Saga style	Choreography	Orchestration (Temporal)	Fewer moving parts at this size; revisit if steps exceed 5
Event publishing	Transactional outbox	Publish directly to Kafka	Avoids the dual-write problem (DB commits, Kafka fails)
Broker	Kafka	RabbitMQ	Replay, partition ordering, high throughput
Cart store	Redis	MySQL	1,700 RPS of small writes; TTL built in
Vector DB	pgvector	Pinecone / Elasticsearch	One less system; 1 M vectors fits easily

Decision	Chosen	Alternative	Why
Consistency	Strong for stock, eventual elsewhere	Strong everywhere	CAP: availability for browsing, correctness for money and stock
Polling for order status	Poll every 2 s	WebSocket / SSE	Simple; 600 RPS is cheap. Upgrade to SSE later

17. Cost Estimate (AWS, peak-capable, monthly, approximate)

Item	Cost (USD)
EKS + 8 × m6i.large (avg)	750
RDS (2 primaries Multi-AZ + 4 replicas)	1,100
ElastiCache Redis (3 × r6g.large)	450
MSK Kafka (3 × m5.large)	500
MongoDB (Atlas M20)	150
CloudFront + S3 + data transfer	300
OpenSearch / monitoring	300
Gemini API (~1.5 M chats)	200–600
Total	**≈ $3,800–4,200 / month**

The learning version runs on a single EC2 t3.large for about $60 a month, or on the free tier with fewer components.

18. Future Improvements

- Multi-region active-passive with Aurora Global Database
- Elasticsearch / OpenSearch for faceted search at > 10 M products
- Orchestrated sagas with Temporal once flows grow
- Personalised recommendations from order history (collaborative filtering)
- CQRS read model for order history
- Service mesh (Istio/Linkerd) for mTLS and traffic shifting

19. Glossary

Term	Meaning
RPS	Requests per second
p99	99% of requests are faster than this
Saga	A sequence of local transactions with compensating actions
Outbox	Store an event in the same DB transaction as the data, publish it later
Idempotent	Processing the same request or message twice has the same effect as once
HPA	Kubernetes Horizontal Pod Autoscaler
RAG	Retrieval-Augmented Generation: give the LLM real data as context
DLT	Dead Letter Topic for messages that keep failing
RPO / RTO	Max data loss / max recovery time after a disaster